

IS597TML
Trustworthy Machine Learning

2. Regression, Logistic Regression, and Deep Learning

Haohan Wang

haohanw@Illinois.edu

Agenda

- Linear Regression and Logistic Regression
- Deep Learning
- Gradient Descent



Linear Regression Intuition

- Linear Regression is to find a straight line that fits the data
  - Advantages
    - Intuitive
    - Fast to compute
    - Explainable
  - Disadvantages
    - Only straight line
      - Limited hypothesis space (representation power)
    - cannot handle high-dimension data
      - High-dimension in the statistical sense
    - explainability assumes independence among features
    - and …

What are the advantages and/or disadvantages of it?

$$\boldsymbol{y} = \mathbf{X}\beta + \epsilon$$

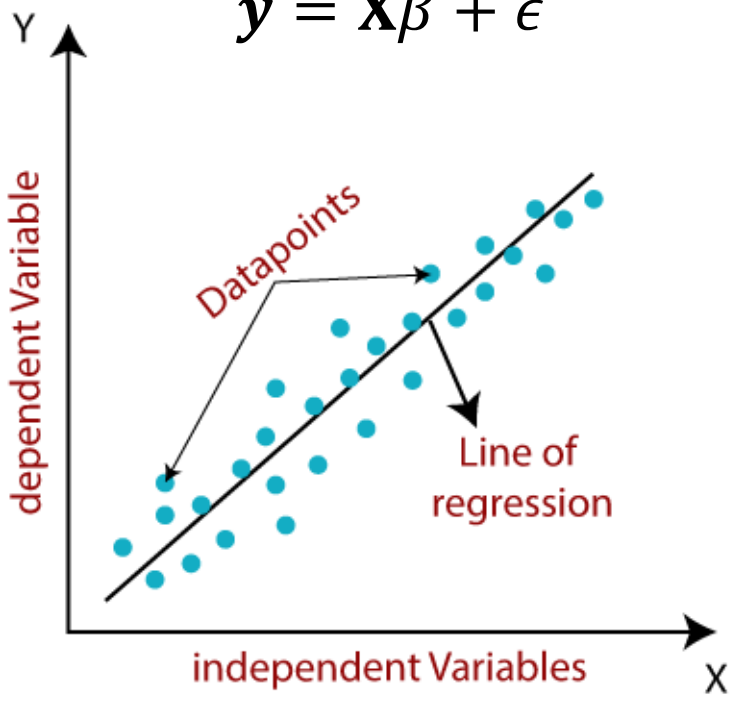


Image source: https://www.javatpoint.com/linear-regression-in-machine-learning

Linear Regression

- Given
  - feature matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$
  - label vector $\boldsymbol{y} \in \mathbb{R}^{n}$
- We aim to fine a regression coefficient β with

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2$$

- Closed-form solution

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{y}$$

Please make sure you are perfectly comfortable with the notations on this slide



What's the issue of this closed-form solution?

The challenges in high-dimension setting

- Solving a simple problem for β, and we denote the result with $\tilde{\beta}$

$$\mathbf{0} = \mathbf{X}\beta$$

- An example when $n = 2, p = 3$:

$$\begin{bmatrix} 0 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 2 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \begin{array}{c} 2y + 4z = 0 \\ y = -2z \\ \\ 2x + y = 0 \\ 2x - 2z = 0 \\ -2z = -2x \\ x = z \end{array} \Rightarrow \tilde{\beta} = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} z$$

The challenges in high-dimension setting

- Then, for a linear regression problem
  - For a solution of

$$\hat{\beta} = \underset{\beta}{\operatorname{argmin}} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2$$

  - as $\hat{\beta}$, then all $\hat{\beta}+ \tilde{\beta}$ is also a solution

So, now, we met a problem, and we need to do something about it

Discussion

- How do you solve this problem?
  - We can say we just use Lasso or Ridge or any variants, but then how to answer the next question?
- In general, when we meet a new problem in ML, what can we do about it?
  - We probably want to do better than “wait for someone else find a method and then use it”

We Reap What We Sow

- What EXACTLY is our problem now?
  - The dataset we work on now has too many features?
    - Hmm, so we change to another dataset?
  - The linear regression cannot generalize well (in terms of prediction error)
    - So, we use a better model?
  - The linear regression cannot deal with this many features
    - So, we figure out a way to tell the model don't work on this many features
  - The closed form solution $\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{y}$ doesn't work in high-dimension
    - So, we fix it with a trick to make it work in high-dimension
  - Any other thoughts?

We Reap What We Sow

- What EXACTLY is our problem now?
  - The dataset we work on now has too many features?
    - Hmm, so we change to another dataset?
  - The linear regression cannot generalize well (in terms of prediction error)
    - So, we use a better model?
  - The linear regression cannot deal with this many features
    - So, we figure out a way to tell the model don't work on this many features
  - The closed form solution $\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{y}$ doesn't work in high-dimension
    - So, we fix it with a trick to make it work in high-dimension
  - Any other thoughts?

Sparsity Regularization

- Our aim is to design a regularization R that regularizes the model to not use that many features
  - Let's just call it sparsity regularization
  - It looks sufficient that R, as a function, only operates on β
  - Then maybe also give it a condition for the output of R to meet

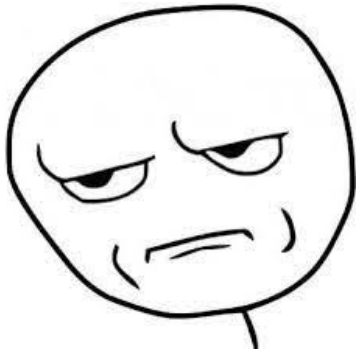
$$\underset{\beta}{\operatorname{argmin}} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 \quad \text{s.t. } \mathrm{R}(\beta) < t$$

- What is the most straightforward regularization?

Sparsity Regularization

- The most straightforward regularization

$$\mathrm{R}(\beta) = \mathrm{count_how_many_features_is_used}(\beta)$$



- The most straightforward but mathematical describable regularization
  - First, we need to answer
    - What does “used” mean in math?
      - How about let’s say β for that feature has non-zero coefficients?
  - Then, all we need is a regularization of how many non-zeros in β

$$\mathrm{R}(\beta) = \|\beta\|_0$$

Sparsity Regularization – subset selction

- Now we have our first solution to the problem

$$\underset{\beta}{\operatorname{argmin}} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 \quad \text{s.t.} \quad \|\beta\|_0 < t$$

- Where t is simply how many features we want



What's the problem of this then?

Sparsity Regularization

- So, $\|\beta\|_0$ cannot be easily, if ever possible, optimized
  - It's just too far from being convex

- What's now?
  - What is the most straightforward, mathematical describable, and convex regularization?

  - What is the closest friend of $\|\beta\|_0$ that is being convex?

Sparsity Regularization

- So, $\|\beta\|_0$ cannot be easily, if ever possible, optimized
  - It's just too far from being convex

- What's now?
  - What is the most straightforward, mathematical describable, and convex regularization?

  - What is the closest friend of $\|\beta\|_0$ that is being convex?
    - How about $\|\beta\|_1$?

Sparsity Regularization - Lasso

- Finally, here is one of the most popular regularization in linear regression family

$$\underset{\beta}{\operatorname{argmin}} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 \quad \text{s.t.} \quad \|\beta\|_1 < t$$

- We usually use this form for the convenience of optimization

$$\underset{\beta}{\operatorname{argmin}} \frac{1}{2}\|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 + \lambda\|\beta\|_1$$

Sparsity Regularization - Lasso

- Lasso regularization
  - Give a sparse set of results
  - Convex, almost differentiable everywhere
  - Widely used in high-dimension linear regression, especially when only a subset of variables are needed
- Other technical details are not covered here
  - Why does Lasso give a sparse set of results?
    - The argument that it looks close to l_0 norm seems quite weak.
  - Lagrange Multiplier
    - The thing that turns the intuitive regularization form to the other
  - How to estimate it parameters?
    - Proximal gradient descent or coordinate descent seems to be the most effective ones

We Reap What We Sow

- What EXACTLY is our problem now?
  - The dataset we work on now has too many features?
    - Hmm, so we change to another dataset?
  - The linear regression cannot generalize well (in terms of prediction error)
    - So, we use a better model?
  - The linear regression cannot deal with this many features
    - So, we figure out a way to tell the model don't work on this many features
  - The closed form solution $\hat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\boldsymbol{y}$ doesn't work in high-dimension
    - So, we fix it with a trick to make it work in high-dimension
  - Any other thoughts?

Inverse a Matrix

- What is the simplest trick to make $\mathbf{X}^T\mathbf{X}$ invertible?
  - How about make it into $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$?
  - Then, we will have a closed form solution

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\boldsymbol{y}$$

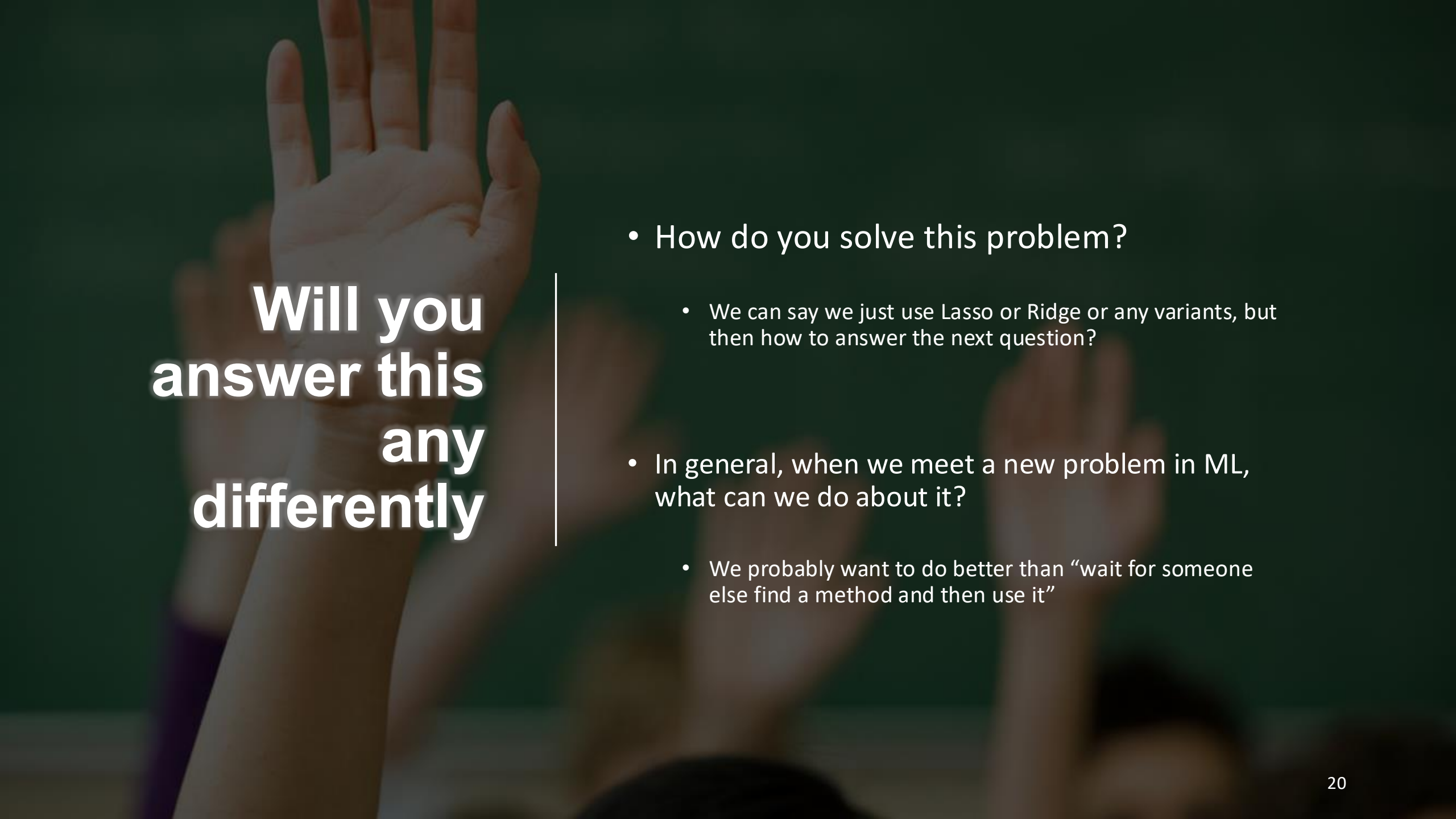
- If this equation is a solution, what is the problem?
  - What's the regularization that leads to the above solution?

Ridge Regularization

- Another popular regularization in the linear regression family

$$\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2$$

- Note:
  - Ridge regularization is not invented or motivated from the above process, here it is just a way to show how we can solve the machine learning problem we met.
  - For a nice read about Lasso and Ridge, one can refer to
    - Tibshirani, R., & Wasserman, L. (2017). Sparsity, the lasso, and friends. Lecture notes from "Statistical Machine Learning," Carnegie Mellon University, Spring.



Will you answer this any differently

- How do you solve this problem?
  - We can say we just use Lasso or Ridge or any variants, but then how to answer the next question?
- In general, when we meet a new problem in ML, what can we do about it?
  - We probably want to do better than “wait for someone else find a method and then use it”

Case Study

- Applying Lasso for Selection of Biomarkers in Genomic Data

Journal of Biomedicine and Biotechnology • 2005:2 (2005) 147–154 • DOI: 10.1155/JBB.2005.147

RESEARCH ARTICLE

Classification and Selection of Biomarkers in Genomic Data Using LASSO

Debashis Ghosh[1] and Arul M. Chinnaiyan[2]

[1]*Department of Biostatistics, University of Michigan, 1420 Washington Heights, Ann Arbor, MI 48109-2029, USA*
[2]*Departments of Pathology and Urology, University of Michigan, 1300 Catherine Road, Ann Arbor, MI 48109-1063, USA*

Received 3 June 2004; accepted 13 August 2004

- The actual technique in the paper is more complicated than we need to discuss today, here we just talk about a simplified version as the preparation for our next topic.

Selecting Important Genes

- 101 samples with 9984 genes
- noncancer vs. cancer tissues as label
  - We will talk about classification later, let's just use regression for it now.
  - So, let's say, instead, our data is about some measurable cancer status
    - Like the expression of some well-known cancer biomarkers
- We build a model $$\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 + \lambda\|\beta\|_1$$
- Solve the above equation to find β
  - The sparsity pattern of the β will tell us which genes are important

Selecting Important Genes

- Results

Clone ID	Gene name
Hs.288965	*Homo sapiens* cDNA: FLJ22300 fis, clone HRC04759
Hs.76307	Neuroblastoma, suppression of tumorigenicity 1
Hs.9615	Myosin, light polypeptide 9, regulatory
Hs.226795	Glutathione S-transferase pi
Hs.171731	Solute carrier family 14 (urea transporter), member 1 (Kidd blood group)



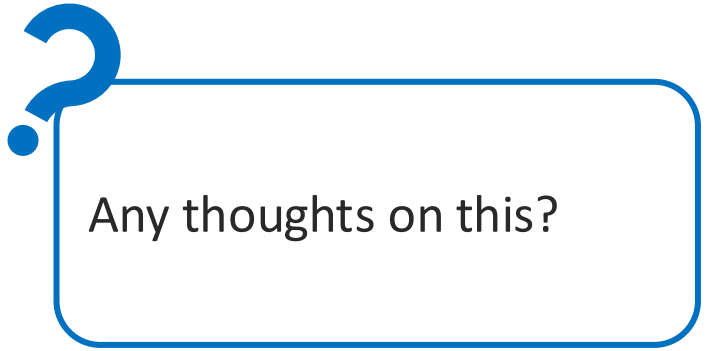
How can we improve this?
(let's say a biologist believes it is important to consider the interaction of genes in the data)

Case Study – Lasso and Its Extensions

- Problem:
  - How to use Lasso to select genes, while genes connected in the same network will affect the disease status in the same way?
- Formulation:
  - Network: $\mathbf{N} \in \mathbb{R}^{p \times p}$
  - feature matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$
  - label vector $\boldsymbol{y} \in \mathbb{R}^{n}$
  - affect the disease status in the same way:
    - We extend this lasso formulation:

$$\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1$$

    - so that the coefficients of β are similar for those genes that are connected through the network



Network Regularized Lasso

- How to define the differences of coefficients for coefficients of gene i and gene j?
  - How about $\left\|\beta_i - \beta_j\right\|$
- So, the final form becomes

$$\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \|\boldsymbol{y} - \mathbf{X}\beta\|_2^2 + \lambda\|\beta\|_1 + \gamma \sum_{i,j} \mathbf{N}_{i,j} \left\|\beta_i - \beta_j\right\|$$

- If you are interested in a more computationally efficient form
  - Li, C., Li, H.: Network-constrained regularization and variable selection for analysis of genomic data. Bioinformatics 24(9), 1175–1182 (2008). ISSN: 1367–4803.
  - Wang, H., Lopez, O. L., Wu, W., & Xing, E. P. (2022). Gene Set Priorization Guided by Regulatory Networks with p-values through Kernel Mixed Model. In International Conference on Research in Computational Molecular Biology (pp. 107-125). Springer, Cham.

Another Example

- A biologist says
  - “you machine learning people are discovering those genes that are easy to identify, but what we are interested in are the ones that are not so easy to find”
- An example:
  - Wang, H., Vanyukov, M. M., Xing, E. P., & Wu, W. (2020). Discovering weaker genetic associations guided by known associations. BMC medical genomics, 13(3), 1-10.



Discussion

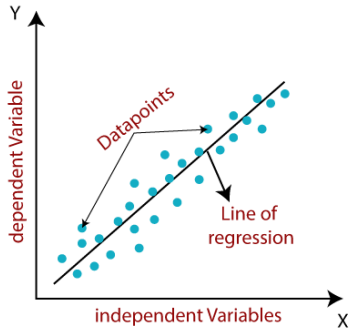
- What do you think is the most important thing so far, today?

A Summary

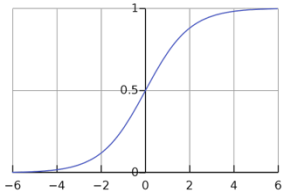
Machine Learning is really this easy
(within the scope of this course, and
if you attend lectures regularly)



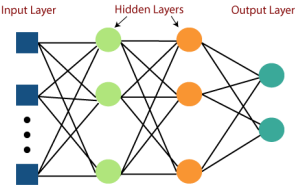
- Understand what you need
- Design a mathematical description of what you need



Regression cannot do classification → Turn continuous into categorical

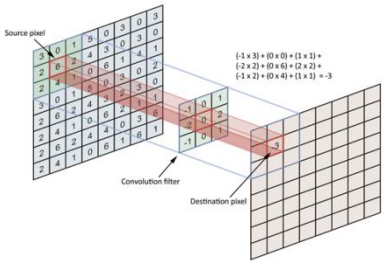


Linear function cannot do much → Introduce non-linearity and going deep





Pixels are not locally independent → convolution



Test distribution is not the same as training → align them

Agenda

- Linear Regression
- Regularization
- Case Study of Lasso
- Logistic Regression
- Multi-layer Perceptron
- Gradient Descent and Its Variants
- Overfitting and Cross-validation

What if what we need is classification

- It seems classification becomes one of the most popular tasks in machine learning
- Let's talk about binary classification first
- Our problem:
  - Linear regression cannot represent the classification task well
  - What do we really need in binary classification?
    - The probabilities of one class given features



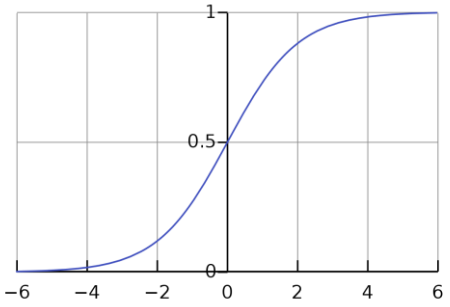
What's the mathematical description of this?

Image: https://dzone.com/articles/machinex-simplifying-logistic-regression

Step Function and Sigmoid Functions

- You probably heard of the logistic function

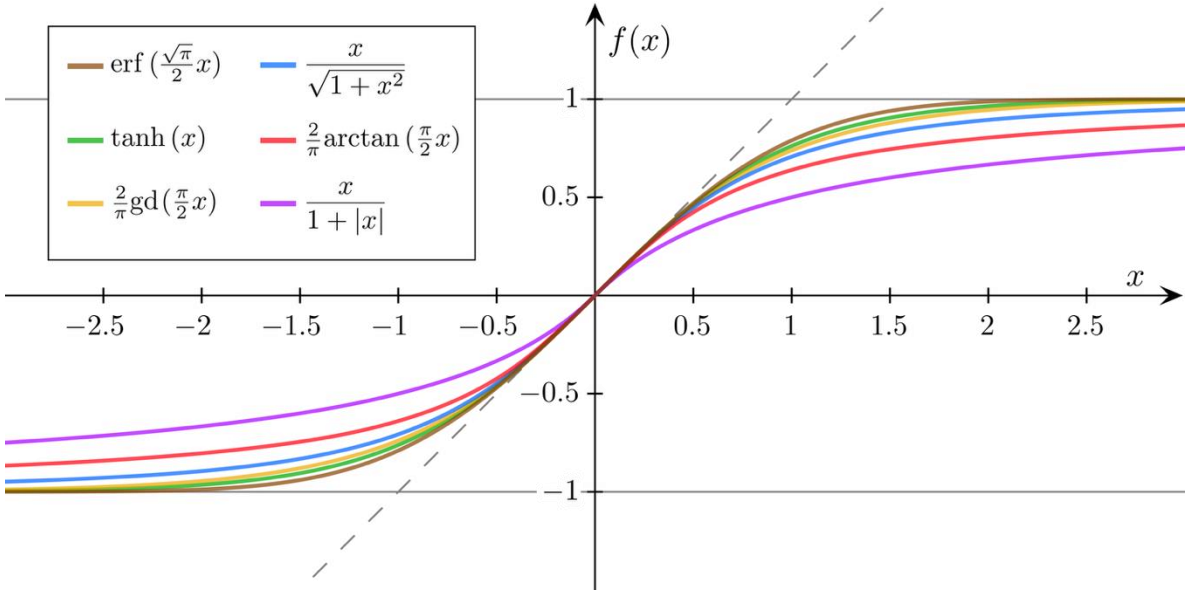
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



  - bounded bounded between 0 and 1
  - differentiable
  - easily computed gradient

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- what's wrong with these guys?



From Linear Regression to Logistic Regression

- Now we have a good function for “the probability of one class”
- So, from the linear regression formulation to the logistic regression

$$\mathrm{f}(\mathbf{x}) = \mathbf{x}\beta \quad \Rightarrow \quad \mathrm{f}(\mathbf{x}) = \sigma(\mathbf{x}\beta)$$

- So, now, instead of $\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \sum_{(\mathbf{x},y)\in(\mathbf{X},\mathbf{y})} \|y - \mathbf{x}\beta\|_2^2$
- Maybe we do $\underset{\beta}{\operatorname{argmin}} \frac{1}{2} \sum_{(\mathbf{x},y)\in(\mathbf{X},\mathbf{y})} \|y - \sigma(\mathbf{x}\beta)\|_2^2$

?

What’s wrong with this?

From Linear Regression to Logistic Regression

- Since the output of the model is a probability, we better treat it like that
- Cross-entropy

$$\mathbf{H}(p, q) = -\mathbb{E}_p[\log(q)]$$

- Loss function in binary logistic regression

$$-\mathbb{E}_{p(y)}\left[\log\left(q(\hat{y})\right)\right]$$
$$= -(\mathrm{p}(\mathrm{y}=1)\log\left(q(\hat{y}=1)\right) + \mathrm{p}(\mathrm{y}=0)\log\left(q(\hat{y}=0)\right))$$
$$= -(y\log\left(\sigma(\mathbf{x}\beta)\right) + (1-y)\log(1-\sigma(\mathbf{x}\beta)))$$

?

Out of so many discrepancies that can measure the differences between distributions, why do we have to choose this one?

Logistic Regression

- Function

$$\widehat{p(y)} = \frac{1}{1 + e^{-\beta x}}$$

- Loss function

$$-(y \log(\widehat{p(y)}) + (1 - y)\log(1 - \widehat{p(y)}))$$

Multi-class Classification

- How do achieve multi-class classification?
  - (at high-level)
  - Turn the model into multi-class classification
  - Turn the problem into binary classification

- One-vs-all classification
  - Build multiple binary classification models, one of each class
  - Combine these classification models together with heuristics

Multi-class Classification Model

- What do we need?
  - The model function maps input data to a vector of probabilities
    - Probabilities must all be non-negative and sum to 1.
- What do we have?
  - Logistic regression model
    - Map the data into one probability
  - Linear regression model
    - Map the data into a continuous output (that we know how to turn into probability)

Multi-class Classification Model

- We start to put linear regression models in parallel

$$\mathrm{f}(\mathbf{x}) = \begin{bmatrix} \mathbf{x}\beta_0 \\ \mathbf{x}\beta_1 \\ \dots \\ \mathbf{x}\beta_2 \end{bmatrix} = \mathbf{x}\mathbf{W} \qquad \mathbf{W} \in \mathbb{R}^{p \times k}$$

- Now we have a function that maps data into a vector of values
  - We turn them in probabilities
  - With something we call softmax function

$$\sigma(\mathbf{x})_j = \frac{e^{\mathbf{x}_j}}{\sum_j e^{\mathbf{x}_j}}$$

About $\sigma(x)$

- If you are interested, try to show that

$$\sigma(\mathbf{x})_j = \frac{e^{\mathbf{x}_j}}{\sum_j e^{\mathbf{x}_j}}$$

- functions the same as the following in binary classification

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- So, from now on, we just call it $\sigma(\mathbf{x})$

Multi-class Classification Model

- Now we have a model that maps data into a vector of probabilities

$$\mathrm{f}(\mathbf{x}) = \sigma\left(\begin{bmatrix} \mathbf{x}\beta_0 \\ \mathbf{x}\beta_1 \\ \dots \\ \mathbf{x}\beta_2 \end{bmatrix}\right) = \sigma(\mathbf{xW}) \qquad \mathbf{W} \in \mathbb{R}^{p \times k}$$

- And a final piece is the loss function
  - Cross-entropy can easily generalize to multi-class settings, so we are good

A summary

	Linear regression	Logistic regression	Multi-class classification
maps data	$\mathbf{x}\beta$	$\mathbf{x}\beta$	$\mathbf{xW}$
turn representations into probability	-	$\sigma(\mathbf{z})$	$\sigma(\mathbf{z})$
Loss function	MSE	Cross-entropy	Cross-entropy

Discussion

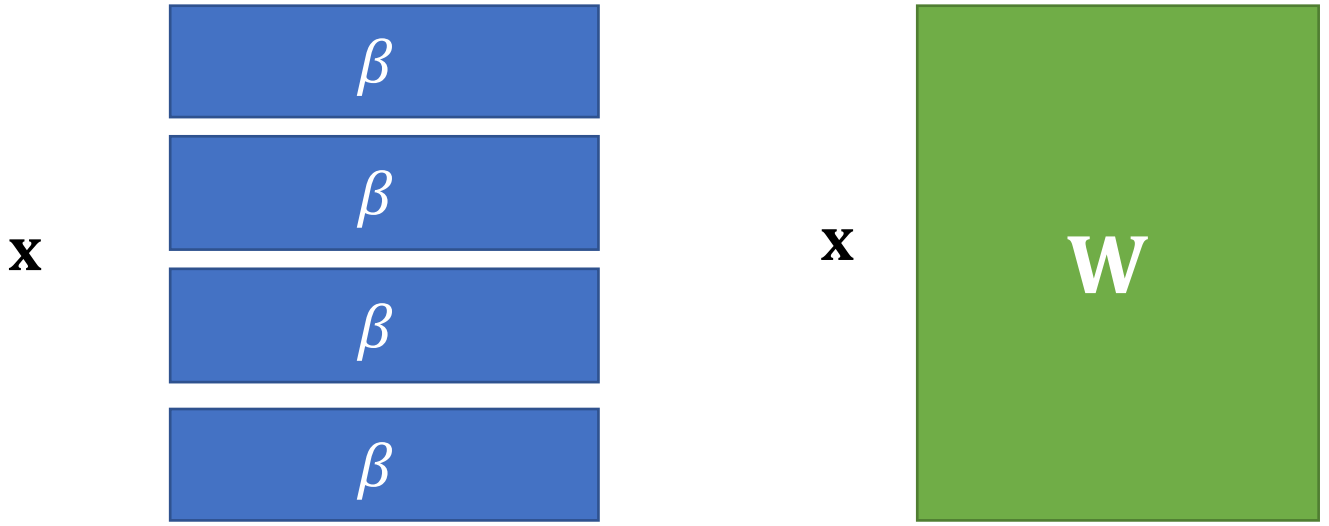
- Following our path from linear regression to Lasso and its variants, do you feel the power to extend multi-class classification to tons of new things?
  - Some examples?
- For these three models, what have we learned that can be considered basic building blocks?
  - Linear mapping, softmax, MSE, cross-entropy
- Out of these basic building blocks, which are invented because of the problem itself, and which are invented because of the computing convenience?
- What innovations can you do if you are equipped with the power to be able to compute anything (we can ignore any computing needs)?
  - Pretty much the era we are at right now

What we have seen so far, in terms of concatenation of weights

- Our starting point $\mathbf{x}\beta$, which maps data into a scalar



- We pile them along one dimension to create the multi-class model





What if we just pile it along the other dimension?

Concatenation of Weights

- If we pile weight along the other direction



- Some stuff to notice now
  - The input-output dimensions of these $\mathbf{W}$ need to be matched for the computation to be feasible
  - However, only the input dimension of the first $\mathbf{W}$ and the output dimension of the last $\mathbf{W}$ are fixed, all the rest are open to adjustments.

Concatenation of Weights

- If we pile weight along the other direction



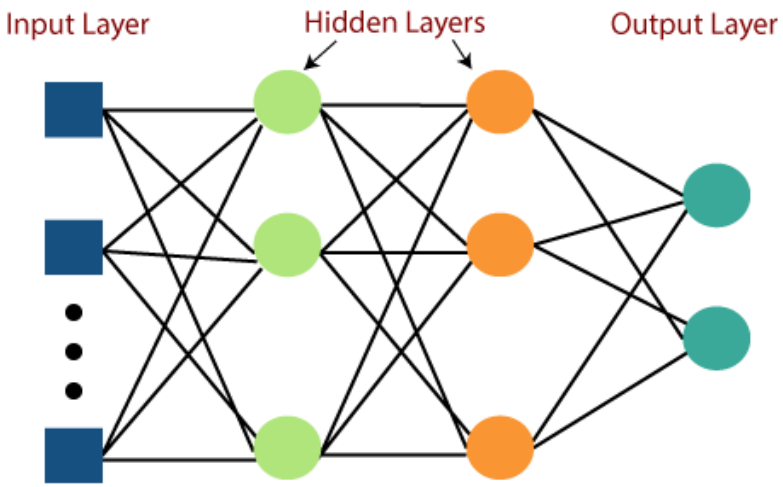
- But this doesn't seem right….

$$\mathbf{x}W_1W_2\,W_3\,\ldots W_m = \mathbf{x}A$$

- So the concatenation seems pointless…

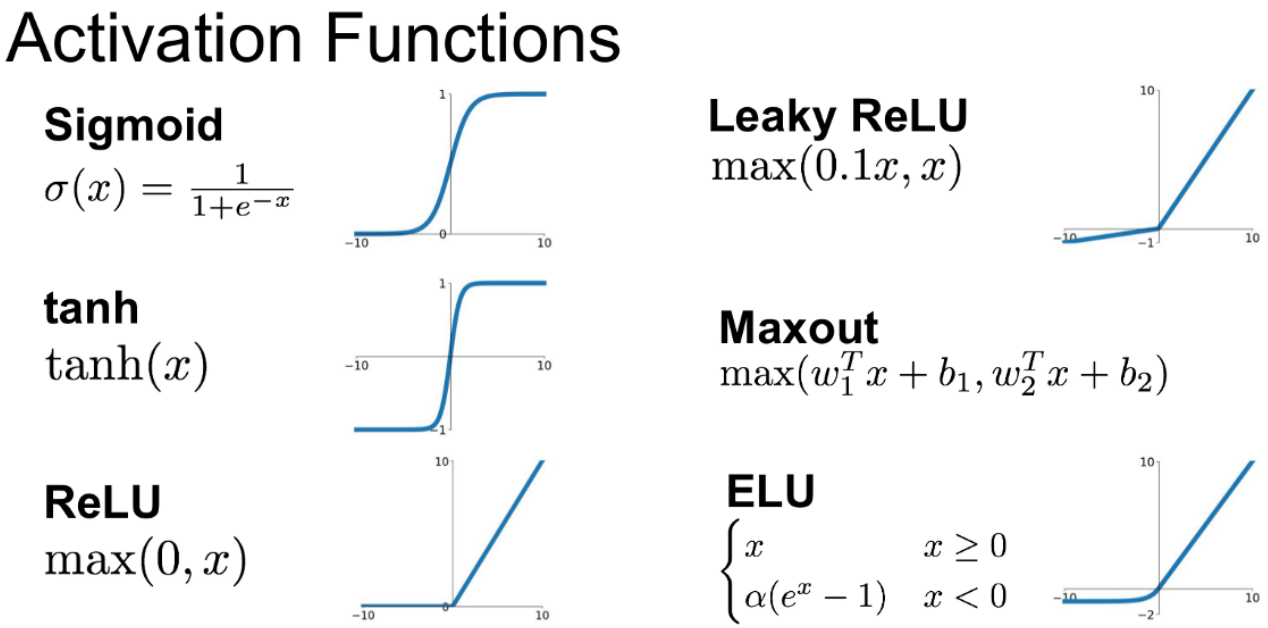
Introducing non-linearity

- But things will be different if we introduce a little bit non-linearity after each weight
  - We define $\mathrm{f}_k(\mathbf{z}) = \boldsymbol{\sigma}_k(\mathbf{z}\mathbf{W}_k)$
  - We define $\mathrm{f}(\mathbf{x}) = \mathrm{f}_m \odot \mathrm{f}_{m-1} \odot \ldots \odot \mathrm{f}_m\,(\mathbf{x})$
    - where we denote $a \odot b(\mathbf{z}) = a(b(\mathbf{z}))$
- A Multiple-layer Perceptron for Classification
  - $\mathrm{f}(\mathbf{x})$
  - Usually outside the final layer, there is a softmax
  - Usually trained with cross-entropy loss



A little more on $\sigma(x)$

- In the context of deep learning, it's usually called activation function





Discussion

- As the first deep learning model we covered, what are the limitations of MLP?
- How would you improve it?

Parameter Estimation

- Now we have a model (weights) and a loss function, how to find the weights the lead to a good solution?
  - What does good solution mean here?
    - The parameters with a small loss
- How to search for it?
  - Gradient Descent
    - Some major questions:
      - Where to go
      - How fast we go

Gradient Descent

- The main idea
- Gradient
  - The direction where the model has the most increments

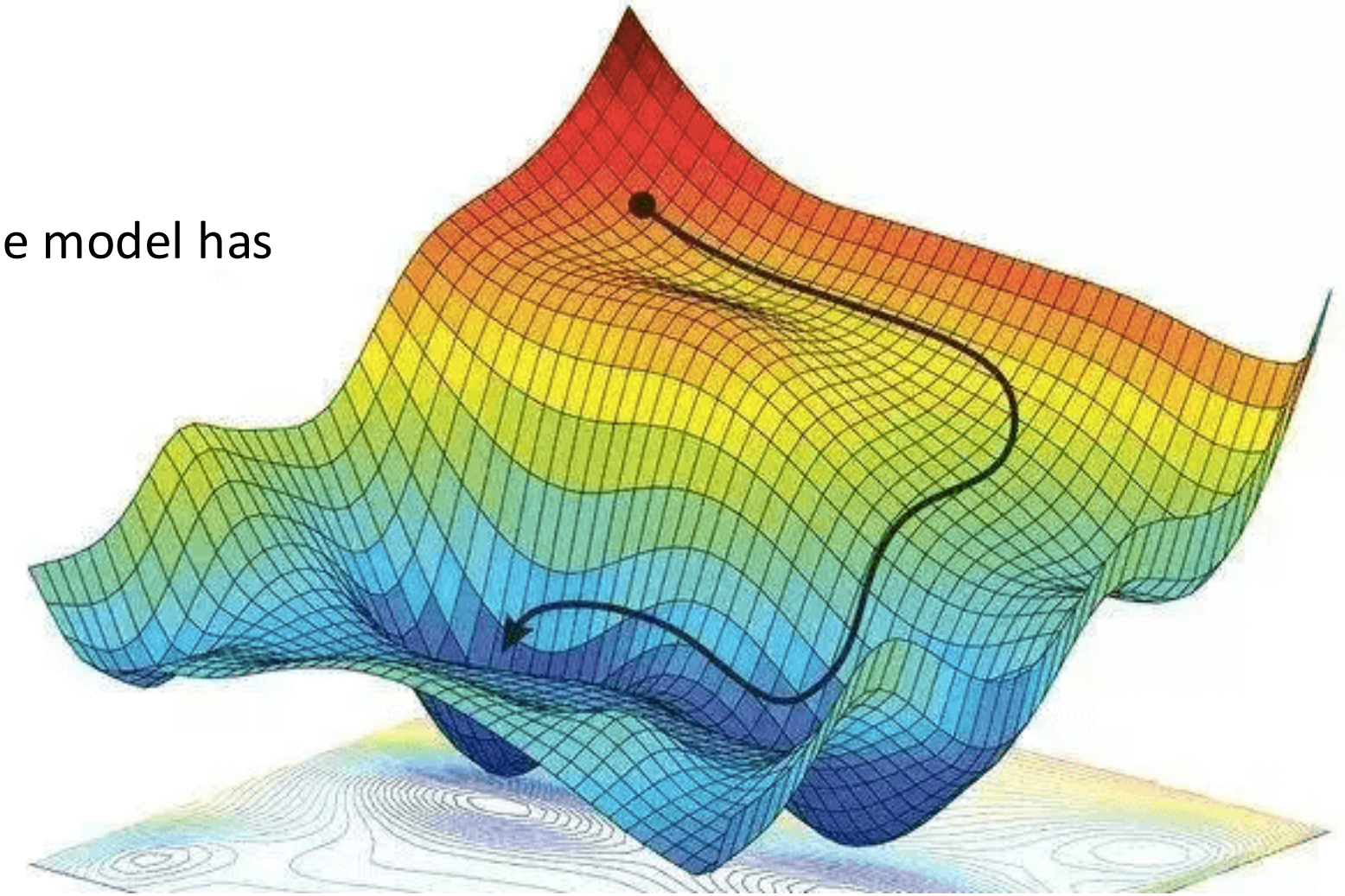


Image https://easyai.tech/en/ai-definition/gradient-descent/

Gradient Descent

- Notations
  - We denote the function as $\mathrm{f}(\mathbf{x}; \mathbf{W})$
  - We denote loss function as $\mathrm{l}(,)$
- Update
  - Start at iteration 0, with random initialization
  - At iteration t, we:
    - calculate the gradient $\partial \mathrm{l}(\mathrm{f}(\mathbf{x}; \mathbf{W}_{t-1}), \mathrm{y}) / \partial \mathbf{W}_{t-1}$
    - update the parameters$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta \partial \mathrm{l}(\mathrm{f}(\mathbf{x}; \mathbf{W}_{t-1}), \mathrm{y}) / \partial \mathbf{W}_{t-1}$
  - Until convergence
- Stochastic gradient descent
  - With iterative batch

Learning Rate

- Learning Rate Choices

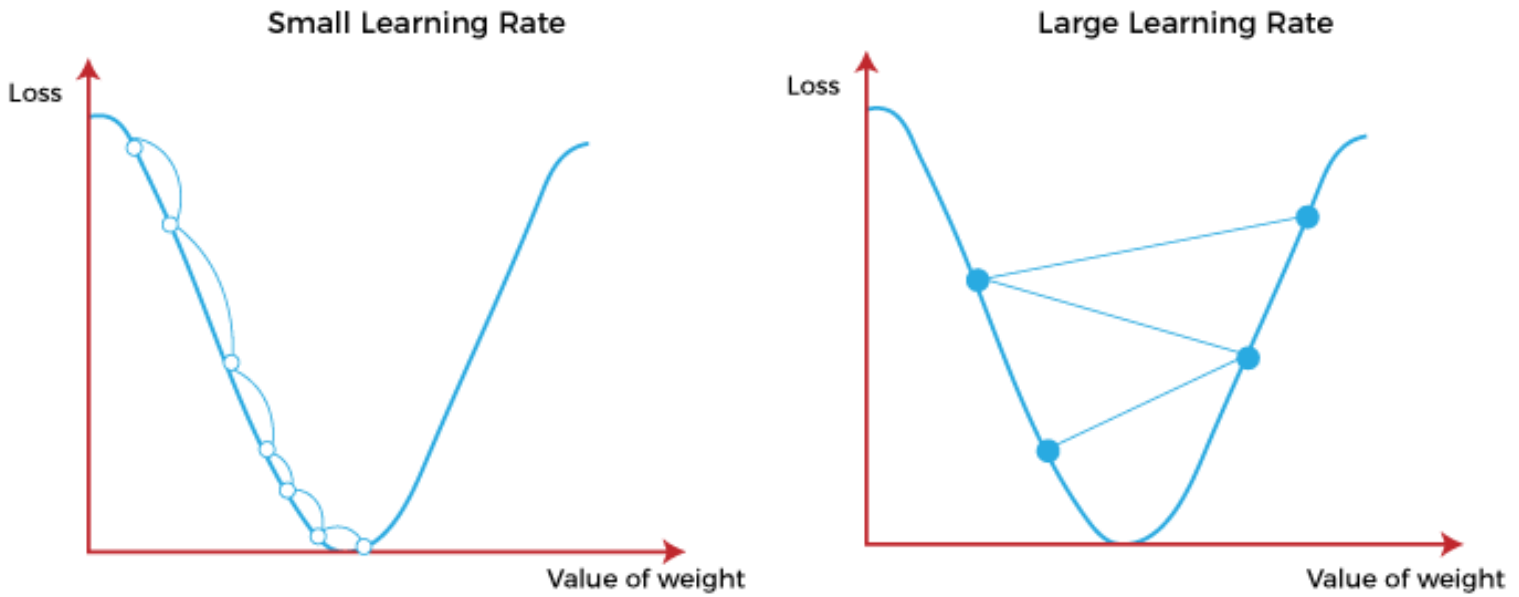


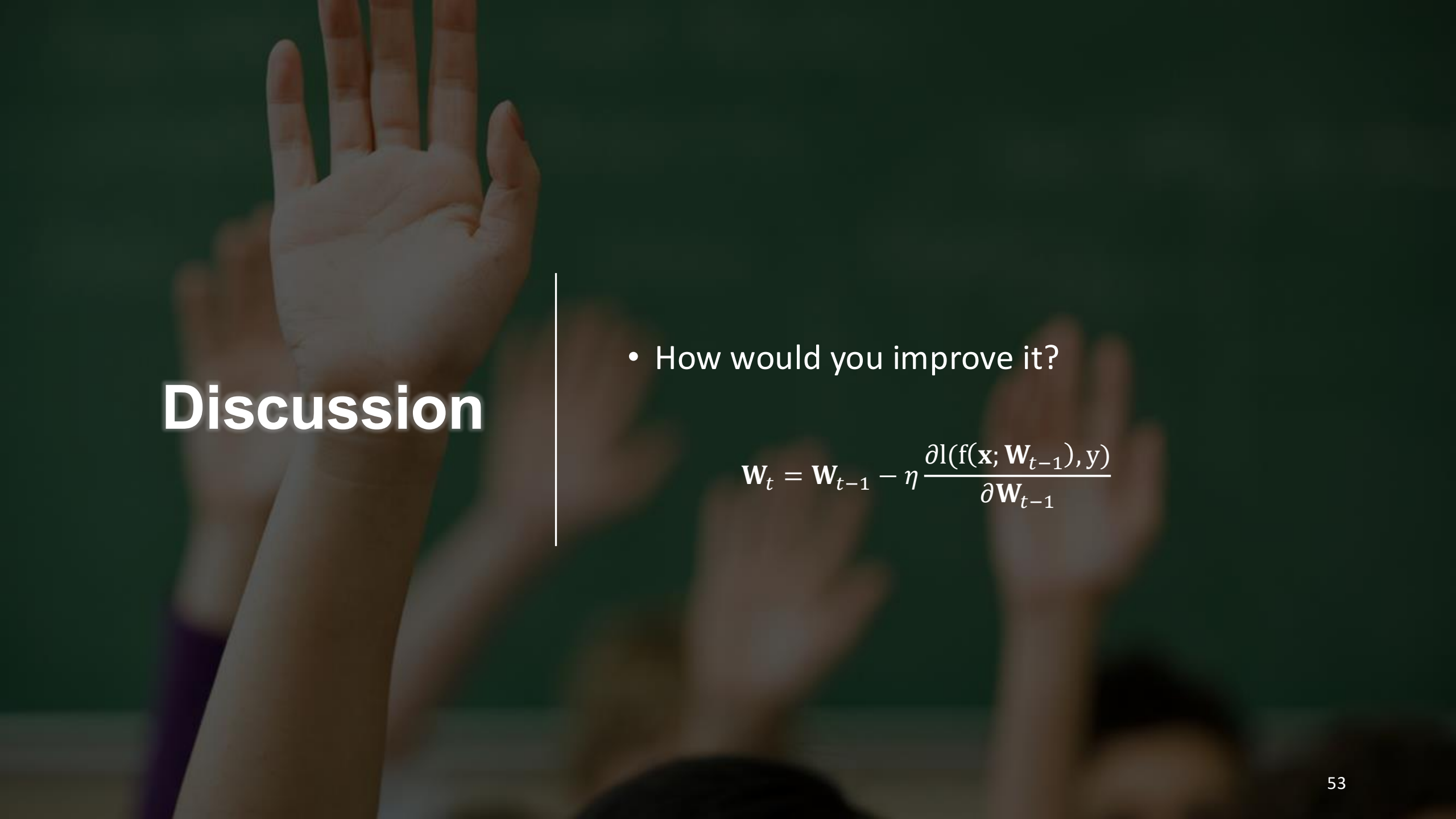
Image https://www.javatpoint.com/gradient-descent-in-machine-learning

Local Optimal vs. Global Optimal

- To avoid getting stuck in local optimal
  - Repeat runs with different initializations
  - Learning rate scheduler
  - …



Is global optimal always a good thing?



Discussion

- How would you improve it?

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta \frac{\partial \mathrm{l}(\mathrm{f}(\mathbf{x}; \mathbf{W}_{t-1}), \mathrm{y})}{\partial \mathbf{W}_{t-1}}$$

Model Selection



- Are you familiar with these terms?
  - Overfitting
  - Cross-validation
    - and what if our data is too big for CV

Model Selection



Can I first do some pre-training of model over the entire dataset before the actual cross-validation process

Model Selection



Why sometimes we have train-test split but other times we have train-validation-test split?

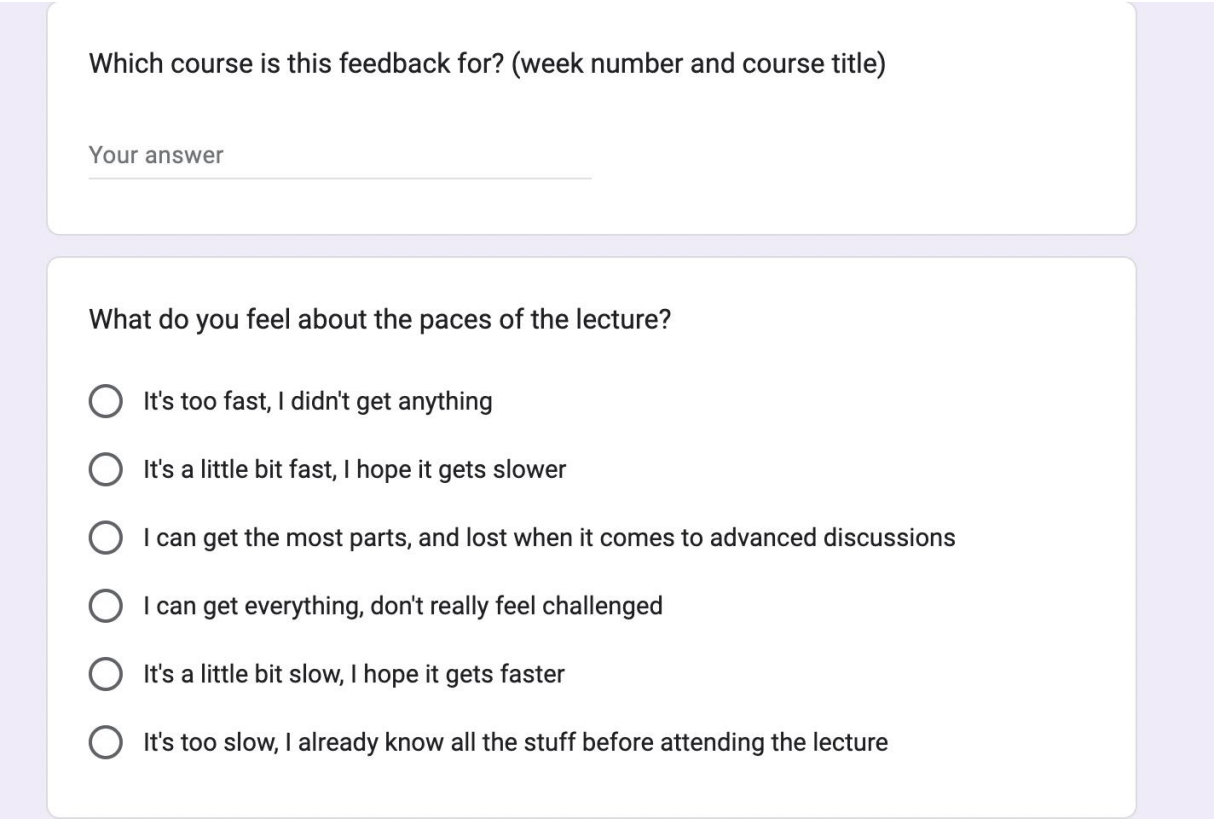
Model Selection



If the official dataset partition is train-test, do I report the highest test accuracy or last-epoch accuracy?

Course Feedback

- Fill in the survey for feedbacks on the paces of the course: https://forms.gle/EgTE82XtadR87eJk9



Which course is this feedback for? (week number and course title)

Your answer

What do you feel about the paces of the lecture?

- ○ It's too fast, I didn't get anything
- ○ It's a little bit fast, I hope it gets slower
- ○ I can get the most parts, and lost when it comes to advanced discussions
- ○ I can get everything, don't really feel challenged
- ○ It's a little bit slow, I hope it gets faster
- ○ It's too slow, I already know all the stuff before attending the lecture

Agenda

- Linear Regression and Logistic Regression
- Deep Learning
- Gradient Descent

